

ParentVeda — Backend Table Blueprint

The full plan of Supabase tables, grouped by build phase. A living reference — not frozen.

Mother side focus · Stack: Supabase (Postgres) · Generated for future reference

How to read this map (4 principles)

1. **auth.users is free.** Supabase auto-creates a hidden `auth.users` table when you turn on auth — it holds email, hashed password, and a unique **user id**. You never build or touch it. Everything else hangs off it.
2. **Almost every table has a `user_id` column** pointing back to that user. That single column lets the security rules (RLS) enforce "this mum sees only her own rows." It is the backbone of the whole design.
3. **One store ≠ always one table.** Some stores *split* into several tables (`tools_store` holds 4 things → 4 tables); some *merge* (`journal_store` + `memory_store` → one table).
4. **Files don't go in tables.** Photos and voice notes live in **Storage buckets**; the table just stores a short text pointer (the file's URL). See the last section.

● Foundation — Phase 1 build first

Just two tables. Together they power login + the due-date-driven week calculation.

Table	Who builds it	What it holds	From your code
<code>auth.users</code>	Supabase (automatic)	email, password (hashed), user id	—
<code>profiles</code>	us	name, role (mother/father), due_date , partner link, created_at	auth flow + <code>pregnancy_controller</code> due date

● Core user data — Phase 2 the meaty stuff (~14)

The genuinely valuable, hard-to-lose entries.

Journal & memories

Table	From store	Holds
<code>journal_entries</code>	<code>journal_store</code> + <code>memory_store</code> (merged)	text / photo / voice / auto entries
<code>bump_photos</code>	<code>bump_store</code>	one bump pic per week + caption + favorite flag

Trackers (your single `tools_store` splits into 4)

Table	Holds
<code>weight_logs</code>	dated weight + note
<code>movement_logs</code>	baby kick / movement counts
<code>contractions</code>	timed contractions + intervals
<code>kegel_sessions</code>	kegel progress

Health

Table	From store	Holds
medications	medicine_store	meds & supplements + schedule
symptom_logs	symptom_store	symptom entries
scans_appointments	scans_store	scans & appointments

Daily & planning

Table	From store	Holds
daily_moods	daily_store	dated mood logs
baby_talk	daily_store	"Dear Baby" vault messages
kept_affirmations	daily_store	saved affirmations
calendar_events	calendar_store	personal events
reminders	reminder_store	customizable reminders

● Saved / bookmarked state — Later small, simple

Mostly tiny "join" tables (`user_id` + an item id + maybe a flag). Low-stakes, so they come after launch.

Table	From store(s)	Holds
reading_progress	read_next + read_done + read_to_baby* (merged)	saved / reading / completed states
saved_videos	video_store	bookmarked videos
can_i_saved	can_i_store	saved "Can I...?" Q&As
garbh_favorites	garbh_store	Garbh Sanskar favorites + reflection tally
expert_follows	expert_follow_store	followed experts
hospital_bag_items	hospital_bag_store (v1+v2 merged)	packing checklist state
journey_dates	journey_dates_store	edited milestone dates

● Shopping — Later ~4

Table	From store	Holds
saved_products	product_store	wishlisted products
bought_products	bought_store	"already bought" markers
cart_items	cart_store	preview cart
product_checklists	product_checklist_store	user-built checklists

● Community — Later most relational, ~4

`community_store` is the richest — it splits into several linked tables.

Table	Holds
<code>community_posts</code>	user posts
<code>community_comments</code>	comments on posts
<code>community_engagement</code>	likes / votes / saves
<code>community_joins</code>	which groups a user joined

 Static content — Phase 4 with the admin panel

These hold *shipped content*, not user data — so **no** `user_id` (everyone reads the same rows). They **stay as JSON assets for now** and only move to the DB when you want to update content without releasing a new app.

Table	Holds	Today
<code>week_content</code>	week-by-week curriculum (<code>weekContent.json</code>)	stays an asset
<code>home_daily_content</code> , <code>garbh_content</code> , <code>can_i_database</code> , ...	the 19 hardcoded Dart data files	stay assets

 Father side — Later out of current scope

Table	Holds
<code>father_journal_entries</code>	father's separate journal (<code>father_journal_store</code>)
<i>pairing tables</i>	real partner-pairing (built with the father side)

 Files (not tables) — Storage buckets

Photos and voice notes are too big for tables, so they live in **Storage buckets**. The matching table just keeps a text URL pointing at them.

Bucket	Holds	Pointer column
<code>journal-media</code>	voice notes & journal photos	<code>journal_entries.media_url</code>
<code>bump-photos</code>	bump pictures	<code>bump_photos.image_url</code>

The honest summary

- **Eventually:** ~30 tables — just your 27 drawers, a few split apart, mapped to the cloud.
- **To launch a solid mother backend:** only the  Foundation +  Core (~16 tables) truly matter.
- **To start right now:** literally 2 — `auth.users` (automatic) and `profiles` .
- **Nothing is frozen.** This is a living blueprint; we merge/split as we build each one.

ParentVeda backend plan · tables are created via SQL in Supabase as each phase begins · every user-data table carries a `user_id` guarded by Row-Level Security.